Degree project in Computer Science and Engineering

First cycle, 15 credits

Comparative Analysis of A* and Q-Learning Algorithms for Robot Path Planning in Dynamic Warehouse Environments

LUCAS KRISTIANSSON

FELIX WINKELMANN

Comparative Analysis of A* and Q-Learning Algorithms for Robot Path Planning in Dynamic Warehouse Environments

LUCAS KRISTIANSSON
FELIX WINKELMANN

Degree Project in Computer Science Engineering, First Cycle 15 credits
Date: July 4, 2025
Supervisor: Jana Tumova
Examiner: Pawel Herman
School of Electrical Engineering and Computer Science
Swedish title: Jämförande analys av A* och Q-learning-algoritmer för robotbanplanering i dynamiska lagerlokaler

Abstract

The purpose of this thesis was to compare the performance of the A* and Q-learning algorithms for robot path planning in dynamic warehouse environments. The primary metric was throughput, defined as the number of packages successfully transported per robot within a set number of simulation steps. Results showed distinct strengths for each algorithm depending on environmental complexity. The Q-learning algorithm, enhanced with Double Deep Q-Network (DDQN) and QMIX for multi-agent coordination, demonstrated superior adaptability and higher throughput in environments with dynamic obstacles such as humans. In contrast, the A* algorithm proved reliable primarily in static or less dynamic scenarios. These findings suggest reinforcement learning methods, when suitably enhanced and tuned, can outperform traditional heuristic algorithms in dynamic warehouse conditions.

Sammanfattning

Syftet med denna avhandling var att jämföra prestandan hos A*- och Q-learning-algoritmerna för planering av robotbanor i dynamiska lagermiljöer. Det primära måttet var genomströmning, definierat som antalet paket som framgångsrikt transporterats per robot inom ett visst antal simuleringssteg. Resultaten visade tydliga styrkor för varje algoritm beroende på miljöns komplexitet. Q-learning-algoritmen, förbättrad med Double Deep Q-Network (DDQN) och QMIX för koordinering mellan flera agenter, visade överlägsen anpassningsförmåga och högre genomströmning i miljöer med dynamiska hinder, såsom människor. Däremot visade sig A*-algoritmen vara tillförlitlig främst i statiska eller mindre dynamiska scenarier. Dessa resultat tyder på att inlärningsmetoder, när de är lämpligt förbättrade och finjusterade, kan överträffa traditionella heuristiska algoritmer i dynamiska lagerförhållanden.

Contents

Chapter 1

Introduction

The evolution of technology has transformed the way people shop, with e-commerce experiencing rapid growth in recent years (Hashemi-Pour, 2023). Customers expect fast delivery, adding pressure to the companies in question. As demand continues to rise, businesses must become more efficient to keep up. One crucial aspect is warehouse management, where optimizing operations is essential. The implementation of warehouse robots can significantly improve efficiency by automating tasks, reducing errors, and accelerating order fulfilment (de Koster, 2018). These robots can be used to pick and process customer orders more quickly and accurately, ensuring faster deliveries and improved customer satisfaction. Additionally, the introduction of robots can help reduce employee costs by minimizing the need for manual labour in warehouses, allowing companies to allocate resources more effectively.

However, for warehouse robots to operate efficiently, they must be programmed with optimized path-finding algorithms that enable them to navigate warehouse environments quickly and safely. The purpose of this thesis is to investigate how different path-planning algorithms compare in terms of throughput and adaptability in dynamic warehouse environments. Specifically, we focus on evaluating and comparing the effectiveness of the A* algorithm, a well-established heuristic search method known for efficiently finding shortest paths, and the Q-learning algorithm, a reinforcement learning technique capable of adapting to dynamic and unpredictable environments.

By analysing the throughput defined as the number of items transported from point A to B of robots utilizing these two algorithms, this research aims to provide insights into their practical suitability for warehouse logistics.

1.1 Warehouse

Order picking is a crucial task in warehouses, directly impacting efficiency and customer satisfaction. Traditionally, manual order picking is performed by humans workers who retrieve items from storage locations and prepare them for shipping. This process involves workers navigating aisles, collecting items, and fulfilling orders based on predetermined lists (orders). Manual order picking is both time-consuming and exhausting. In addition, this process accounts for a significant portion of warehouse costs (de Koster et al., 2007).

Order picking can be done with different strategies based on the layout of the warehouse and the item selection, the following strategies are common (CognitOps, 2023).

Piece Picking: This type of picking involves picking each item individually from an order. This picking type is used by e.g. E-commerce businesses like Amazon because the orders are often small and a mix of products.

Batch Picking: This type of picking involves grouping multiple orders with common items, it is then possible to pick more than one object per storage destination. This minimizes the travel distance since multiple orders will be picked simultaneously. This picking type is used by e.g. supermarkets for online grocery orders, where employees gather common items for multiple customers simultaneously.

Zone Picking: This picking type involves dividing the warehouse into multiple zones and limiting the pickers to specific zones. These pickers will pick items in their own zone, and then the picked items are consolidated into an order. This picking style is used by e.g. Auto parts warehouses, as it effectively divides items into zones based on size, creating a streamlined picking strategy.

No matter the picking strategy a warehouse employs, human error is always a possibility when people are involved—such as selecting the wrong items or quantities. Additionally, as previously mentioned, order picking can be physically demanding, requiring workers to walk long distances and lift heavy items. Moreover, it is a major expense, accounting for approximately 65% of total warehouse costs. To address these challenges, many companies have invested in robots to handle this costly process, reducing expenses while also protecting the health of their workers (Lee et al., 2019).

1.2 Robots

There are several ways robots can be used in warehouses to enhance the efficiency of order picking.

One approach is the Basket System, where automated vehicles carry a pick cart that follows the human worker throughout the picking process. Acting as a mobile basket, the robot moves alongside the worker, collecting items as they are picked. Once the order is complete, the robot autonomously transports the basket to a delivery station, while a new robot with an empty basket takes its place. This method optimizes the worker's walking path, minimizing unnecessary movement and increasing efficiency (de Koster, 2018).

Another strategy is the Robotic Movable Rack (RMR) System, where robots transport entire storage racks containing multiple items. When an item is requested, a robot retrieves the rack holding the item and brings it to a stationary human worker, who then picks the required product and places it into an order bin. In this setup, the human remains in one place while the robots handle all transportation, significantly reducing physical strain and improving productivity (de Koster, 2018).

Some robots are capable of handling the entire order-picking process independently. They can navigate the warehouse, pick items into a basket, and transport the completed order to its delivery station. By automating these tasks, these robots can effectively replace human pickers, improving efficiency and reducing labour costs (Brightpick, 2024).

1.2.1 Path planning for Warehouse Robots

Integrating robots into warehouses can boost operational efficiency, largely by optimizing how they navigate and transport items. A critical challenge here is **path planning**, the task of determining the best routes for robots to quickly reach their destinations. This task becomes even more challenging when the environment changes dynamically with human workers and other robots are continuously moving around.

In this thesis, we specifically focus on robots that perform **Piece Picking**, where each robot is tasked with transporting individual items from randomly assigned pickup points to designated drop-off locations. This scenario is highly relevant in modern e-commerce shipping centres.

Understanding how different path-planning algorithms perform under these conditions provides valuable insights into their practical suitability for real-world warehouse environments.

1.3 Research Question

Given this context, our study aims to answer the following research question: How do the A* algorithm and Q-learning algorithm compare in terms of throughput, defined as the number of items transported per unit of time when utilized by robots for path planning in a dynamic warehouse environment?

1.4 Scope and Limitations

This thesis specifically investigates the comparative performances of A* and Q-learning algorithms in a simulated discrete gridworld environment that replicate real-world warehouse conditions. The primary metric for comparison is throughput, focusing on efficiency in navigating and completing transportation tasks.
Available information between the two evaluated algorithms:

- The A* algorithm computes the robot's paths through a centralized planner that holds global knowledge of the static warehouse layout (such as shelves) and current positions of other robots. However, it does not possess global visibility of dynamic obstacles, such as human workers. Each robot have a 5x5 grid vision (with cameras, sensors, etc.) around it that is used to avoid collision with dynamic obstacles.
- In contrast the Q-learning agents operate purely on the 5x5 observations of their immediate surroundings, without direct global knowledge during execution. However, these agents were trained centrally, where global information was available to facilitate coordination and learning.

This thesis does not address the following aspects:

- Economic analyses such as cost-benefit calculations or ROI assessments.
- Detailed mechanical or hardware-related performance concerns, such as energy consumption, robot reliability, sensors, cameras, or maintenance needs.
- Real-world experimental validation or deployment in operational warehouses.

These limitations define clear boundaries for this research, ensuring a focused exploration of algorithmic performance within controlled simulation scenarios.

1.5 Thesis Outline

This thesis is structured to provide a comprehensive analysis of path planning algorithms for warehouse robotics. The following chapters present a logical progression from problem identification to solution evaluation:

Chapter 1: Introduction
Presents the research context, motivation, and objectives. Establishes the importance of efficient path planning in modern warehouse operations and formulates the research question comparing A* and Q-learning algorithms.

Chapter 2: Background
Provides the theoretical foundation for the study. Reviews existing path planning algorithms, explains the fundamental concepts of A* and Q-learning, and discusses centralized versus decentralized approaches.

Chapter 3: Methods
Describes the research methodology, including:

- Simulation environment design
- Implementation details of both algorithms
- Experimental setup and evaluation metrics

Chapter 4: Results
Presents the experimental findings, including throughput measurements, performance comparisons, and statistical analysis across different warehouse scenarios.

Chapter 5: Discussion
Analyzes and interprets the results, explaining the implications for warehouse robotics. Also addresses:

- Ethical considerations related to automation and workforce displacement
- Sustainability aspects of robotic warehouse systems
- Energy consumption and environmental impact

Chapter 6: Conclusion
Summarizes the key findings, discusses their practical implications, and suggests directions for future research.

Bibliography

Provides the complete list of references used in the work.

Appendix

Contains technical specifications and detailed implementation information for researchers seeking to replicate or extend this work.

Chapter 2

Background

2.1 Algorithms

A fundamental aspect of robotics is path planning. Path planning algorithms determine the optimal route for a robot to navigate from its starting point to its destination. There are several path planning algorithms, each with its own strengths and weaknesses. This section provides a brief overview of common path planning algorithms, followed by a detailed discussion of A* and Q-learning, which are the focus of this study.

2.1.1 Common Path Planning Algorithms

A* Algorithm: The A* algorithm is a popular path planning algorithm that uses a heuristic to estimate the cost of reaching the goal from a given point. It is widely used in robotics due to its efficiency and accuracy in finding the shortest path. The A* algorithm is based on Dijkstra's algorithm but incorporates a heuristic to guide the search process, making it more efficient (Hart et al., 1968).

RRT: The Rapidly-exploring Random Tree (RRT) algorithm is a probabilistic path planning algorithm that is well-suited for high-dimensional spaces. It works by incrementally building a tree of random samples from the configuration space, connecting them to the nearest node in the tree. The RRT algorithm is particularly useful for robots with complex kinematics or environments with obstacles, as it can efficiently explore the space and find feasible paths (LaValle, 1998).

D* Lite: The D* Lite algorithm is an incremental path planning algorithm that is well-suited for dynamic environments. It works by updating the path

incrementally as new information becomes available, allowing the robot to adapt to changing conditions. The D* Lite algorithm is particularly useful for robots operating in environments with moving obstacles or changing terrain, as it can quickly update the path to avoid collisions and reach the goal efficiently (Koenig et al., 2002).

Swarm Intelligence: Swarm intelligence algorithms are inspired by the collective behaviour of social insects, such as ants and bees. These algorithms work by simulating the interactions between individual agents to find optimal solutions to complex problems. Swarm intelligence algorithms are particularly useful for multi-robot systems, as they can coordinate the actions of multiple robots to achieve a common goal. Some common swarm intelligence algorithms include Ant Colony Optimization (ACO), which mimics the foraging behavior of ants, and Particle Swarm Optimization (PSO), which simulates the social behaviour of birds flocking or fish schooling (Dorigo et al., 2006; Kennedy et al., 1995).

Reinforcement Learning: Reinforcement learning is a machine learning technique that enables robots to learn optimal policies through trial and error. By interacting with the environment and receiving feedback on their actions, robots can learn to navigate complex spaces and perform tasks efficiently. Reinforcement learning is particularly useful for robots operating in dynamic environments, as they can adapt their behaviour based on changing conditions. Common reinforcement learning algorithms include value-based methods like Q-learning and Deep Q Networks (DQN), as well as policy-based methods like Advantage Actor-Critic (A2C) and Proximal Policy Optimization (PPO). Each approach has different strengths and limitations in terms of stability, sample efficiency, and computational requirements (Sutton et al., 2018; Schulman et al., 2017).

2.1.2 Centralized vs. Decentralized Algorithms

Centralized Algorithms: Centralized algorithms involve a single centralized entity making decisions for all robots in the system. This approach is simple and efficient, as the centralized entity can coordinate the actions of all robots to achieve a common goal. However, this approach can be vulnerable to single points of failure, as the centralized entity may become a bottleneck if it is overloaded or fails. In addition, centralized algorithms may not scale well to large numbers of robots, as the centralized entity may struggle to coordinate the actions of all robots effectively.

Decentralized Algorithms: Decentralized algorithms involve individual

robots making decisions autonomously based on local information. This approach is more robust and scalable than centralized algorithms, as each robot can make decisions independently without relying on a centralized entity. Decentralized algorithms are well-suited for multi-robot systems, as they can adapt to changing conditions and operate effectively in dynamic environments. However, decentralized algorithms may be less efficient than centralized algorithms, as robots may not coordinate their actions optimally to achieve a common goal.

2.1.3 Centralized Training with Decentralized Execution

Centralized Training with Decentralized Execution (CTDE) is a hybrid approach that combines the benefits of centralized and decentralized algorithms. In CTDE, a centralized entity trains the robots in the system using a global model, whereafter the robots execute their action autonomously based on local information. This gives the robots the flexibility to adapt to changing conditions while still benefiting from the more efficient coordination provided by centralized training. CTDE is well-suited for multi-robot systems, as it can leverage the advantages of both centralized and decentralized algorithms to achieve optimal performance (Lowe et al., 2017).

Application in this study: In this study, we apply CTDE to both A* and Q-learning algorithms, since the warehouse environment is a dynamic and complex space that requires robots to adapt to changing conditions while still coordinating their actions effectively. Therefore, we have the following:

- A* with CTDE: Since A* isn't trained, we've made an interpretation of this to Centralized computation of paths, decentralized execution with local adjustments.
- Q-learning with CTDE: Centralized learning from shared experiences, decentralized decision-making based on learned policies.

2.1.4 A* Algorithm

Overview: The A* algorithm is the best-first search algorithm that finds the shortest path from a start node to a goal node in a weighted graph. It was first described by Peter Hart, Nils Nilsson, and Bertram Raphael in 1968 (Hart et al., 1968). A* combines the strengths of Dijkstra's algorithm (completeness and optimality) with the efficiency of a heuristics-guided approach. This

makes it particularly well-suited for path planning in our warehouse environment, where the robots need to navigate efficiently between storage locations and around obstacles to pick items and fulfill orders.

A* details: The core principle of the A* algorithm is to minimize the total estimated cost of a path in a graph network from the start node to the goal node. For each node n in the graph, A* maintains two key values:

- $g(n)$: The actual cost of the path from the start node to node n.
- $h(n)$: The heuristic function that estimates the cost of the path from node n to the goal node.

The total estimated cost, denotes as $f(n)$, is calculated as:

$$f(n) = g(n) + h(n) \tag{2.1}$$

The A* algorithm uses a priority queue to explore nodes in order of increasing $f(n)$ value. At each step, the algorithm selects the node with the lowest $f(n)$ value and expands it by considering its neighbours. The algorithm terminates when the goal node is reached or when the priority queue is empty.

It is described as seen in Algorithm 1.

The heuristic function $h(n)$ is crucial to the performance of the A* algorithm. It provides an estimate of the cost of the path from node n to the goal node. Common heuristics in grid-based environments include:

- Manhattan distance: $h(n) = |x_n - x_{goal}| + |y_n - y_{goal}|$
- Euclidean distance: $h(n) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$
- Chebyshev distance: $h(n) = \max(|x_n - x_{goal}|, |y_n - y_{goal}|)$

For a heuristic to guarantee that A* finds the optimal path, it must be admissible, meaning it never overestimates the actual cost to reach the goal. Additionally, if the heuristic is consistent (or monotonic), meaning that for any node n and its neighbour m, $h(n) \leq c(n, m) + h(m)$, where $c(n, m)$ is the cost of moving from node n to node m, then A* is guaranteed to find the optimal path with the fewest number of expansions (Hart et al., 1968).

Algorithm 1 A* Algorithm

```
1: Initialize openSet with the start node
2: Initialize closedSet as empty
3: For the start node, set g(start) = 0
4: For the start node, set f(start) = g(start) + h(start)
5: while openSet is not empty do
6:     current = node in openSet with lowest f value
7:     if current is the goal node then
8:         Reconstruct path and return success
9:     end if
10:    Remove current from openSet
11:    Add current to closedSet
12:    for each neighbor of current do
13:        if neighbor in closedSet then
14:            Continue to next neighbor
15:        end if
16:        tentativeG = g(current) + cost(current, neighbor)
17:        if neighbor not in openSet then
18:            Add neighbor to openSet
19:        else if tentativeG >= g(neighbor) then
20:            Continue to next neighbor
21:        end if
22:        parent(neighbor) = current
23:        g(neighbor) = tentativeG
24:        f(neighbor) = g(neighbor) + h(neighbor)
25:    end for
26: end while
27: Return failure (no path found)
```

2.1.5 Q-learning Algorithm

Overview: Q-learning is a model-free reinforcement learning algorithm that enables agents to learn optimal action policies through trial and error. It was first introduced by Chris Watkins in 1989 (Watkins, 1989). It has since then become a foundational algorithm in the field of reinforcement learning, known for its simplicity and effectiveness in solving a wide range of problems. In our warehouse environment, Q-learning can be used to train robots to adapt to changing conditions and learn optimal policies for navigating the warehouse, picking items, and fulfilling orders.

Reinforcement Learning Basics: Reinforcement learning is a type of machine learning that enables agents to learn optimal policies by interacting with an environment and receiving rewards or penalties based on their actions. The goal of reinforcement learning is to maximize the cumulative reward over time by learning the optimal policy that maps states to actions. In the context of our warehouse environment, the robots are the agents, the warehouse is the environment, and the rewards are based on the efficiency of the robots' actions, such as picking items and fulfilling orders.

Reinforcement learning is framed as a Markov Decision Process (MDP), defined by:

- States S: The set of all possible states the agent can be in.
- Actions A: The set of all possible actions the agent can take.
- Transition function $T(n'|n, a)$: The probability of transitioning from one state to another given an action.
- Reward function $R(n, a, n')$: The reward received by the agent for taking an action in a given state.
- A discount factor $\gamma \in [0, 1]$: Determines the importance of future rewards.

The goal of the agent is to learn a policy $\pi : S \rightarrow A$ that maximized the expected cumulative discounted reward:

$$E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1})\right] \tag{2.2}$$

Q-learning detailed explanation: The Q-learning algorithm is centred around updating a Q-table that stores the estimated values of all state-action

pairs. The Q-value of a state-action pair (s, a) represents the expected cumulative reward of taking action a in state s and following the optimal policy thereafter. The Q-value is updated iteratively using the Bellman equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R(s,a) + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{2.3}$$

Where:

- α is the learning rate, determining the impact of new information on the Q-value.
- $R(s, a)$ is the immediate reward of taking action a in state s.
- γ is the discount factor, determining the importance of future rewards.
- s' is the next state after taking action a in state s.
- $\max_{a'} Q(s', a')$ is the maximum Q-value of all possible actions in state s'.

The Q-learning algorithm iteratively updates the Q-values based on the rewards received by the agent, gradually learning the optimal policy for navigating the environment and maximizing the cumulative reward. The algorithm continues to explore the environment while exploiting the learned policy, balancing between exploration and exploitation to find the optimal policy.

As pseudo code, the Q-learning algorithm can be described as seen in algorithm 2.

Exploration vs. Exploitation: Early in training, the robot must explore randomly to discover rewarding behaviours. Later, it should exploit what it's learned. The ϵ-greedy policy smoothly manages this transition:

$$a = \begin{cases} \text{random action} & \text{with probability } \epsilon_t \\ \arg\max_a Q(s,a) & \text{otherwise} \end{cases} \tag{2.4}$$

where ϵ_t decays over time, typically from 1.0 (pure exploration) to 0.01 or lower (mostly exploitation). This mimics how humans gradually shift from broad experimentation to refined expertise.

Scalability Challenges in Tabular Q-learning: In a warehouse environment, even a modest 34×32 grid combined with dynamic elements like human workers, other robots, and task-specific states (e.g., carrying status and goal locations) creates a state space so large that storing a traditional Q-table

Algorithm 2 Q-learning Algorithm

Initialize Q(s,a) arbitrarily for all $s \in S$, $a \in A$
Set hyperparameters: learning rate $\alpha \in [0, 1]$ and discount factor $\gamma \in [0, 1]$
for each episode **do**
    Initialize state s
    while s is not terminal **do**
        Choose action a from s using policy derived from Q (e.g., ϵ-greedy)
        Take action a, observe reward r and next state s'
        Update Q-value:
        $Q(s, a) \leftarrow Q(s, a) + \alpha \left[R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a)\right]$
        $s \leftarrow s'$
    end while
end for

becomes impossible. The number of possible states grows exponentially with each added variable, a phenomenon known as the curse of dimensionality. This explosion isn't just about memory; it means the robot would need to visit each state-action pair an impractical number of times to learn anything useful.

Deep Q-learning (DQN) as a solution: The solution is to replace the Q-table with a neural network (a Deep Q-Network or DQN) that learns to approximate Q-values (Mnih et al., 2013, 2015). Instead of storing every possible $Q(s, a)$ combination, the network $Q(s, a; \theta)$ with weights θ takes our 10×5×5 observation tensor, encoding the robot's local surroundings, goals, and status, and outputs Q-values for all possible actions. The network isn't just a lookup table; it learns to generalize from similar states, making the problem tractable. We can express this approximation of the optimal action-value function mathematically as:

$$Q^*(s, a) = \max_{\pi} \mathbb{E}\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots | s_t = s, a_t = a\right] \tag{2.5}$$

which is the maximum sum of rewards r_t discounted by γ at each time step t, achievable by a policy $\pi = P(a|s)$, after making an observation s and taking an action a.

We train this network by minimizing the difference between its predictions and the ideal target Q-values, which combine the immediate reward with the discounted future rewards the robot can expect. Mathematically, we express this as minimizing the Bellman error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q(s', a'; \bar{\theta}) - Q(s, a; \theta)\right)^2\right] \tag{2.6}$$

Here, $\bar{\theta}$ represents a separate target network whose weights are periodically synchronized with the main network. This separation stabilizes training by preventing the network from chasing its own tail, updating its predictions against moving targets. However, this approach suffers from instability due to correlated updates and moving targets.

Stabilizing DQNs with Experience Replay: If we trained the network on consecutive experiences as the robot moves through the warehouse, the strong correlations between nearby states would destabilize learning. Instead, we store all experiences (s, a, r, s') in a replay buffer $\mathcal{D}$, then sample them randomly in mini-batches (Lin, 1992). This is akin to how humans learn better from shuffled flashcards rather than reading a textbook line by line. Some experiences are more informative than others. When the network's prediction is wildly wrong (high temporal difference error δ_i), that transition deserves more attention:

$$p_i = |\delta_i| + \epsilon, \quad \text{where} \quad \delta_i = r + \gamma \max_{a'} Q(s', a'; \bar{\theta}) - Q(s, a; \theta) \tag{2.7}$$

By sampling these high-error experiences more frequently, we accelerate learning, much like how focusing on difficult problems improves our understanding faster than reviewing what we already know. This is called prioritized experience replay (Schaul et al., 2015).

The Overestimation Problem and DDQN: A subtle issue arises because the same network both selects and evaluates actions through the $\max$ operator. This leads to systematically overoptimistic Q-values. Double DQN (DDQN) fixes this by decoupling the two steps:

1. Use the main network θ to select the best action for the next state: $a^* = \arg\max_{a'} Q(s', a'; \theta)$
2. Use the target network $\bar{\theta}$ to evaluate the action's value: $Q(s', a^*; \bar{\theta})$

The new Q-learning update rule becomes:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\left(r + \gamma Q(s', a^*; \bar{\theta}) - Q(s, a)\right) \tag{2.8}$$

This simple change results in more accurate value estimates and better-performing policies (Hasselt et al., 2016).

Target Network Update Methods: The target network is updated less frequently than the main network to stabilize training. Two common methods are:

- **Hard Update:** Copy the weights of the main network to the target network every N steps.
$$\bar{\theta} \leftarrow \theta \tag{2.9}$$
- **Soft Update:** Gradually blend the weights of the main network into the target network.
$$\bar{\theta} \leftarrow \tau\theta + (1 - \tau)\bar{\theta} \tag{2.10}$$
where τ is a small constant (e.g., 0.0005).

We went with soft updates (also referred to as polyak averaging) for our implementation, as they provide a smoother transition and can help stabilize training.

Partially Observable Markov Decision Processes: In a warehouse environment, the robot may not have access to the full state. For example, it may not know the exact locations or states of other robots or human workers. This leads to a partially observable Markov decision process (POMDP) scenario, where the robot must make decisions based on incomplete information. We define the following components for our POMDP:

- States S: The complete state of the environment (which the agent cannot fully observe).
- Actions A: The set of actions available to the agent.
- Transition function $T(s'|s, a)$: State transition probabilities.
- Observation space Ω: The set of possible observations the agent can receive.
- Observation function $O(o|s', a)$: The probability of observing o given the new state s' and action a.
- Reward function $R(s, a)$: The expected reward for taking action a in state s.
- Discount factor γ: Determines the importance of future rewards.

An observation o_t at time t is a partial view of the complete state s_t, which is not fully observable. The agent must maintain a belief state $b(s)$ over the possible states, which is a probability distribution over the states given the history of observations and actions. In RL, this is often approximated using a recurrent neural network (RNN).

In our implementation, with our local observations being a 10×5×5 tensor, we used a technique called frame stacking (Mnih et al., 2013) to create a history of the last 8 observations, simply concatenating the last 8 observations. The shape of the tensor becomes 80×5×5, and it allows the robot to maintain a belief state over the last 8 time steps. Since our environment isn't super complex, this was sufficient to maintain a belief state. In more complex environments, we would need to use a recurrent neural network (RNN) to maintain the belief state over time. Our warehouse scenario is specifically a Decentralized POMDP (Dec-POMDP), where multiple agents (robots) operate in a shared environment, each with its own local observations and actions. The goal is to learn a joint policy that maximizes the cumulative reward for all agents while considering the partial observability of the environment.

Multi-agent Reinforcement Learning: When we consider the case of more than one robot in the warehouse, we enter the realm of multi-agent reinforcement learning (MARL). In this case, each robot is an independent agent that learns its own policy while interacting with other agents. The challenge here is that the actions of one agent can affect the rewards and states of other agents, leading to a complex and dynamic environment. This involves issues like credit assignment, where an agent must determine which of its actions led to a particular outcome, and coordination, where multiple agents must work together to achieve a common goal. There's also the issue of non-stationarity, where the environment is constantly changing due to the actions of other agents. This can make it difficult for an agent to learn a stable policy.

MARL problems can be considered the intersection of reinforcement learning and game theory. The agents can be cooperative, competitive, or a mix of both. In our warehouse environment, the robots are primarily cooperative, as they work together to fulfill orders and navigate the warehouse efficiently. In game theory, this type of environment would be classified as a partially observable stochastic game (POSG), which POMDPs are a special case of (Albrecht et al., 2024).

Centralized vs Decentralized Learning: In MARL, we differentiate between centralized and decentralized learning:

- **Centralized Learning:** All agents share a common Q-table or neural network, allowing them to learn from each other's experiences. This

can lead to faster convergence and better performance, but it requires communication between agents and can be computationally expensive.

- **Decentralized Learning:** Each agent learns its own Q-table or neural network independently. This is more scalable and allows for greater flexibility, but it can lead to slower convergence and suboptimal policies due to the lack of shared information.
- **Centralized Training with Decentralized Execution:** During training, we utilize a centralized Q-table or neural network, allowing all agents to learn from each other's experiences. However, during execution, each agent uses its own Q-table or neural network to make decisions. This approach combines the benefits of both centralized and decentralized learning, leading to faster convergence and better performance while maintaining scalability.

In this work, we primarily focus on the third approach.

Value Decomposition for Multi-agent Cooperation: A key challenge in cooperative MARL is the credit assignment problem, where agents must determine which of their actions contributed to the overall reward. Value decomposition addresses this by breaking down the joint action-value function into individual value functions for each agent. This allows each agent to learn its own value function while still considering the actions of other agents.

VDN (Value Decomposition Networks) is a popular approach for value decomposition in MARL. It uses a neural network to learn the joint action-value function and decomposes it into individual value functions for each agent. The VDN framework is based on the idea that the joint action-value function can be expressed as the sum of the individual action-value functions:

$$Q_{tot}(s, \mathbf{a}) = \sum_{i=1}^{n} Q_i(s_i, a_i) \tag{2.11}$$

where $Q_{tot}(s, \mathbf{a})$ is the joint action-value function, s is the state, $\mathbf{a}$ is the joint action taken by all agents, and $Q_i(s_i, a_i)$ is the individual action-value function for agent i. This allows each agent to learn its own value function while still considering the actions of other agents. However, VDN assumes that the joint action-value function is a simple sum of the individual action-value functions, which may not always be the case in complex environments, for example, if there's limited space in the warehouse and the robots are competing for space (Sunehag et al., 2018).

QMIX: Monotonic Value Factorization: QMIX (Rashid et al., 2020) extends VDN by allowing for more complex interactions between individual and team values, while still permitting decentralised execution. It uses a neural network to learn a mixing function that combines the individual action-value functions into a joint action-value function. QMIX enforces a monotonicity constraint:

$$\frac{\partial Q_{tot}}{\partial Q_i} \geq 0, \forall i \in \{1, \ldots, n\} \tag{2.12}$$

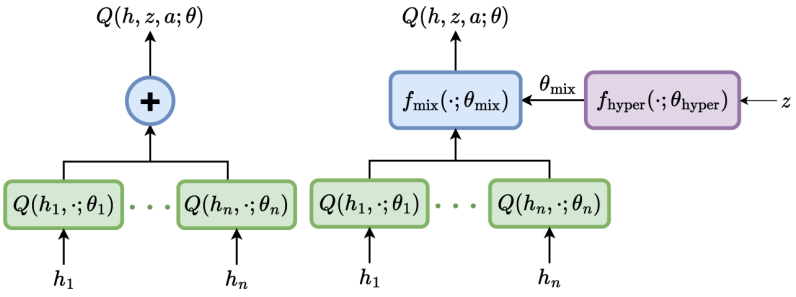
This ensures that the joint action-value function is a non-decreasing function of the individual action-value functions, meaning that if an agent's action-value function increases, the joint action-value function will also increase. Mathematically, this is expressed as:

$$\arg\max_{\mathbf{a}} Q_{tot}(s, \mathbf{a}) = (\arg\max_{a_1} Q_1(s_1, a_1), \ldots, \arg\max_{a_n} Q_n(s_n, a_n)) \tag{2.13}$$

QMIX implements this constraint using a mixing network that combines the individual agent values in a monotonic fashion:

$$Q_{tot}(s, \mathbf{a}) = f_\theta(Q_1(s_1, a_1), ..., Q_n(s_n, a_n), s) \tag{2.14}$$

where f_θ is a mixing network with non-negative weights generated by hypernetworks that take the global state as input. This architecture allows agents to make decisions solely based on their local observations, the mixing network to capture non-linear relationships between agents, and satisfies the monotonicity constraint.



(a) VDN (b) QMIX

Figure 2.1: VDN and QMIX architecture. From Albrecht et al., 2024.

Parameter Sharing: Our agents are homogeneous, meaning they all have the same tasks, behaviours, and optimal policy. This allows us to share parameters between agents, reducing the number of parameters we need to train and

speeding up training. We can use a single neural network for all agents, with the same weights for each agent (Albrecht et al., 2024).

2.2 Related work

Research on path planning for multiple robots in warehouse environments has explored both classical algorithms and reinforcement learning (RL). A* and its variants are commonly used for finding efficient paths in grid-based layouts and are known for performing with near optimal results without any collisions. However, these methods can struggle under high traffic or dynamic conditions due to increased computational demands. Some studies have modified A* to avoid conflicts over time, but purely heuristic approaches tend to perform worse in more complex, real-life scenarios.

Many recent works have turned to RL in order to make reliable path finding algorithms in dynamic environments. Some works even compare A* with deep Q-network (DQN) policies in single-agent tasks. For example, K. Li et al., 2024 showed that DQN agents trained to navigate cluttered warehouse environments outperformed A* in terms of task success rate, accuracy, and F1 score. They also implemented a hybrid method called Proximal Policy-Dijkstra (PP-D), which combined RL with classical planning and achieved even better results. However, their evaluation focused on single-agent navigation rather than throughput in a multi-agent system, and they did not use centralized training with decentralized execution (CTDE).

Other recent studies, such as Y. Wang et al., 2025, combine multi-agent RL with large neighbourhood search to improve coordination. Their hybrid algorithm used RL in early stages to resolve collisions, then switched to prioritized A* to complete the remaining paths. The method achieved high success rates in dense scenarios where classical solvers failed. However, this approach differs from ours, which distinctly compares pure implementations of A* and Q-learning without blending the methods, thereby isolating their individual strengths and weaknesses.

In summary, previous work shows that RL-based planners can outperform classical algorithms like A* under certain conditions, especially in dynamic and multi-agent settings. Combining the two (the hybrid version) have showed good results by combining the strengths of both approaches. Our thesis builds on these findings by providing a direct comparison between the classical A* algorithm and the RL approach, focusing specifically on throughput. Unlike most prior studies, we use a CTDE framework and the PettingZoo multi-agent

environment to evaluate how learning-based methods affect efficiency in a realistic warehouse setting.

Chapter 3

Methods

3.1 Overview

The goal of comparing A* and Q-learning in this thesis is to determine which algorithm more effectively optimizes robot path planning in dynamic warehouse environments. The evaluated metric is throughput, the number of items successfully transported per unit of steps.

3.2 Environment Setup

In order to simulate a realistic warehouse environment for evaluating pathfinding algorithms, we developed a custom multi-agent simulation using the PettingZoo library, which is a Python library designed specifically for reinforcement learning in multi-agent systems. The PettingZoo library provides standardized interfaces for interacting with multi-agent environments and supports several API styles. In our case, we chose the Parallel API, implemented by subclassing pettingzoo.utils.ParallelEnv.

3.2.1 PettingZoo

PettingZoo's ParallelEnv interface allows multiple agents to act simultaneously at each step, which is crucial in our case where multiple warehouse robots and humans operate concurrently. The core methods such as environment initialization, state resetting, step transitions, rendering, and defining agent-specific action and observation spaces were implemented following PettingZoos's standard interface.

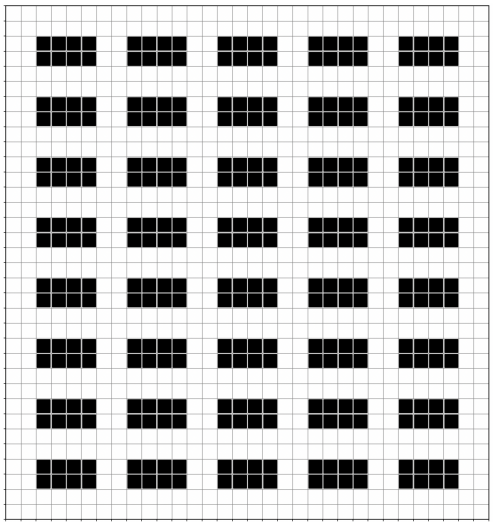


Figure 3.1: Warehouse grid (size 34x32)

3.2.2 Environment Layout

The warehouse is modeled as a two-dimensional grid. The grid size is adjustable depending on the scenario, but in our case we used a default size of 34×32 cells. Shelves are implemented as **static obstacles** in this grid, and their arrangement is designed to resemble a realistic warehouse setup with evenly spaced aisles to support agent navigation.
Each shelf block consists of:

- 2 rows in height (vertical size)
- 4 columns in width (horizontal size)

This means every shelf occupies a 2×4 rectangular area in the grid. The shelves are placed in a repeating pattern throughout the grid, with 2-cell spacing between shelf blocks both vertically and horizontally. This creates clear aisles for agents to move through. As shown in Figure 3.1, the result is a structured and consistent layout that resembles warehouse shelving rows. Additionally, at least a 2-cell-wide empty border surrounds the grid to give agents space to maneuver around the edges. This kind of layout introduces realistic constraints that make the path planning problem more challenging. Agents must find efficient routes to navigate around shelves, avoid collisions, and reach pickup and dropoff points — which is exactly the kind of scenario we wanted to simulate for comparing A* and Q-learning.

Shelves are placed in the grid using a straightforward block-based approach. Starting from the top-left corner, shelf blocks of size 2×4 are added row by row, with spacing in between to form aisles. The placement continues until the specified number of shelf cells has been reached, resulting in a structured and navigable warehouse layout.

In the warehouse environment, human workers are modeled as **dynamic obstacles** that move independently of the robot agents. Their navigation is controlled by a greedy heuristic algorithm that moves each human toward a randomly assigned goal. This adds dynamic complexity to the environment and forces the robot agents to adapt to changing obstacles. Each human is assigned a random goal location on the grid, and once the human reaches the goal, a new goal is automatically assigned. For instance, a robot might be on its way to a drop-off location when a human worker suddenly stops or blocks the aisle; the robot then has to either wait or re-plan its path in order to avoid a collision. Similarly, if two robots approach the same intersection at the same time, they must coordinate (either via the planner or learned policy) to decide how to avoid a collision. These examples illustrate the kind of dynamic path-planning challenges that the algorithms must solve in our simulated warehouse environment.

The human path planning algorithm determines the next move for each human agent in the warehouse simulation. For each human, the algorithm retrieves its current position and goal. If the movement history is uninitialized, it is set up; if a cycle is detected (i.e., the human repeatedly revisits the same positions), a new random goal is assigned and the history is reset. The algorithm then computes the preferred movement by comparing the horizontal and vertical distances to the goal and selects a primary and secondary action accordingly. Crucially, if the selected action is blocked by a robot (as indicated by the `is_valid` function returning a blocking signal), the human is set to wait. This ensures that the robot must adjust its path, rather than the human. The pseudocode 3 summarizes the process.

Algorithm 3 Human Path Planning Algorithm

for each human in Humans **do**
    $current \leftarrow$ current position; $goal \leftarrow$ current goal
    if history not initialized **then**
        Initialize history
    end if
    if cycle detected in history **then**
        Assign new random goal; reset history
    end if
    $action \leftarrow$ ComputeAction($current$, $goal$)
    Update history with $current$
end for
return mapping of humans to actions
function ComputeAction($current$, $goal$)
    if $current = goal$ **then**
        Assign new random goal
    end if
    $dx \leftarrow (goal_{\text{col}} - current_{\text{col}})$
    $dy \leftarrow (goal_{\text{row}} - current_{\text{row}})$
    if $|dx| > |dy|$ **then**
        $primary \leftarrow$ (`RIGHT` if $dx > 0$, else `LEFT`)
        $secondary \leftarrow$ (`UP` if $dy < 0$, else `DOWN`)
    else
        $primary \leftarrow$ (`UP` if $dy < 0$, else `DOWN`)
        $secondary \leftarrow$ (`RIGHT` if $dx > 0$, else `LEFT`)
    end if
    if $primary$ is valid and avoids backtracking **then**
        return $primary$
    else if $primary$ is blocked by a robot **then**
        return `WAIT`
    else if $secondary$ is valid and avoids backtracking **then**
        return $secondary$
    else if $secondary$ is blocked by a robot **then**
        return `WAIT`
    else
        return first alternative valid action, or `WAIT` if none
    end if
end function

3.2.3 Agents' local observation

Each agent receives a local observation in the form of a `(10, 5, 5)` tensor, representing a 5×5 grid centred around the agent. Each of the 10 channels in this tensor contains binary values (0 or 1) and encodes the following information:

1. **Agent position**: A 1 is placed at the center cell to indicate the agent's own position.
2. **Other agents**: Positions of other agents within the 5×5 window are marked with 1.
3. **Shelves (static obstacles)**: Cells that contain shelf blocks are marked with 1.
4. **Humans (dynamic obstacles)**: Cells occupied by human workers are marked with 1.
5. **Current goal**: If the agent's current goal (either a pickup or dropoff point) is visible within the window, it is marked with a 1.
6. **Pickup points**: All visible pickup points within the local observation area are marked with 1.
7. **Dropoff points**: Dropoff points in the 5×5 area are indicated by 1.
8. **Carrying status**: If the agent is currently carrying an item, the entire channel is filled with 1; otherwise, it contains only 0s.
9. **Valid pickup/dropoff indicator**: This channel highlights whether the agent is currently at a valid pickup or dropoff location based on its task progress. For example, if an agent that is carrying an item is standing on the correct dropoff cell, that cell is marked with a 1.
10. **Goal Compass**: This channel tells the agent in which direction the goal is located.

This observation structure ensures agents receive all relevant information about their local surroundings, including task-relevant context and dynamic obstacles, while keeping the state representation compact and scalable. The same state representation is used in both A* and RL. An example of the observation space is illustrated in Figure 3.2.

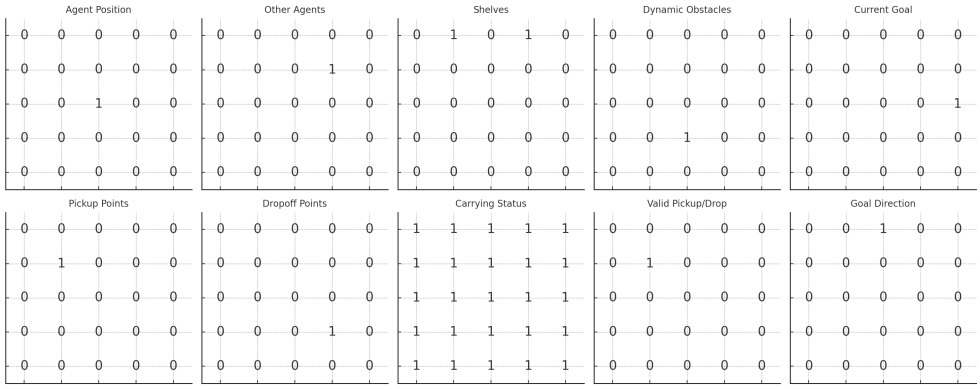


Figure 3.2: Observation-space (5x5 x 10)

3.3 A* implementation

Our A* path planning module implements an analogue to the CTDE (Centralized Training, Decentralized Execution) approach through a centralized planner that computes routes for all robots while enabling decentralized execution. The planner operates on a time-expanded state graph $G = (V, E)$ where:

- Vertices V represent space-time states: $v = (p, t)$ where:
  - $p = (r, c)$ is a position in the warehouse grid
  - t is the time step
- Edges E represent valid transitions: $e = ((p_1, t), (p_2, t + 1))$ where:
  - Movement actions connect adjacent positions
  - Wait actions connect the same position over time
  - All transitions increment time by one step
- Edge costs incorporate:
  - Base movement cost of 1.0
  - Increased wait penalties based on consecutive waits
  - Collisions with reserved space-time points

The planner maintains a global view of this graph, including the warehouse layout and agent states, allowing it to compute collision-free paths simultaneously for every robot. Each robot then independently executes its assigned path, using local observations to adapt to unexpected obstacles.

Multi-Agent Coordination: Planning for multiple robots simultaneously requires careful collision avoidance, which our implementation achieves through prioritized planning and space-time reservations. At the start of each planning cycle, the system determines a priority order for path computation. Robots carrying items receive highest priority since delays in their delivery routes should be minimized, followed by robots actively moving toward goals, and finally idle or at-goal robots.

The planner then computes paths sequentially according to this priority order. As each robot's path is calculated, its trajectory is recorded in a reservation table that tracks cell occupancy over time. When planning subsequent robot paths, the algorithm treats these reserved space-time coordinates as blocked - if a cell is reserved by another agent at time t, no other robot may occupy that cell at the same time. This mechanism prevents simultaneous arrival collisions and allows for coordinated movement through the warehouse.

We also explicitly handle swap collisions, where two robots might attempt to exchange positions in a single step. In such cases, the planner forces one robot to wait, integrating these wait actions into the A* state expansion as additional neighbour states. To maintain efficiency, wait actions incur a slightly higher path cost, ensuring they are only used when necessary for collision avoidance.

Dynamic Obstacle Handling: Our warehouse environment includes human workers that act as unpredictable dynamic obstacles. Rather than attempting to fully model these in the global plan, we combine initial path planning with real-time local adjustments. The initial planning phase considers only static obstacles (shelves) and other robots (via the reservation system). Each robot then uses its local 5×5 grid observation to detect and react to humans and unexpected obstacles during execution.

When a robot detects that its next planned move would lead to a collision - for instance, if a human worker has stepped into its path - it can temporarily deviate from the plan. A local conflict detector compares each robot's intended next position with its current observation, triggering adaptive behaviour when conflicts arise. The robot first attempts to find an alternative direction around the obstacle; if no immediate detour is possible, it inserts a wait action. These adjustments happen without requiring global replanning, and robots resume their original paths once local obstacles clear.

Deadlock Prevention: To prevent robots from becoming permanently stuck, the system actively monitors for deadlock situations. If a robot remains in a wait state for longer than a configured threshold, it triggers a global replanning phase. The planner then recomputes paths for all robots from their current po-

sitions, often resolving the deadlock by finding alternative routes or adjusting movement priorities. Similar replanning occurs whenever a robot completes a task or receives a new goal, ensuring paths remain current and efficient.

This implementation creates a robust path planning system specifically tailored for multi-agent warehouse operations. It leverages the optimality of classic A* pathfinding while trading some of the optimality for practical adaptations for coordination and dynamic obstacle avoidance. The time-aware path planning and reservation system enable conflict-free scheduling, while the CTDE approach provides the adaptability needed for real-world warehouse conditions.

3.4 Q-learning implementation

The Q-learning approach in this thesis refers to a deep reinforcement learning method based on a Double Deep Q-Network (DDQN). To support multi-agent coordination, we further extend the implementation with a QMIX mixing network, enabling centralized training with decentralized execution (CTDE). These enhancements are crucial for handling the complexity and partial observability of dynamic warehouse environments.

Specifically, the following components are integrated into our Q-learning setup:

- Double Deep Q-Network (DDQN)
- Frame stacking
- Prioritized Experience Replay
- Soft Target Updates
- QMIX

3.4.1 Frame Stacking

All of our robots have an observation space consisting of the 10 channels as mentioned before. However in the Q-learning approach we also give information about changes over time, we use a technique called "frame stacking". In our implementation, frame stacking is added as a wrapper around our PettingZoo environment. Instead of providing just a single observation at each time step, we stack the most recent eight observations together. The reason is that a single frame (or state) by itself might not be enough. For example, a robot's

snapshot might show a moving obstacle at a certain position, but from one frame alone the agent can't tell if that obstacle is moving towards it or away from it (Mnih et al., 2013). By stacking the last few frames/states together as the input state, we preserve a short history of what just happened. This gives the agent information about motion and changes over time, kind of like letting the agent see the previous steps as a video.

Frame stacking is a simple way to make the environment more observable without adding complex memory mechanisms. It was initially used effectively in the original DQN paper by Mnih et al. (2013) to play Atari games, where understanding the movement direction of objects, like the ball in Pong, required multiple consecutive frames.

3.4.2 Prioritized Experience Replay

Training a reinforcement learning agent in our warehouse involves a lot of trial and error. The agent stores its experiences where each experience consists of a state, the action taken, the reward received, and the next state (s, a, r, s') in a replay memory. If we sample these experiences uniformly at random (as in a standard experience replay), the agent might spend equal time reviewing both important and not so important experiences. To avoid this, we use a prioritized replay. This technique prioritizes the sampling towards experiences that have more learning value.

The learning value is measured by looking at the temporal difference error (TD error) for each experience, which is basically how surprising or wrong the agent's prediction was for that step. If the agent gets a certain reward and outcome that is very far away from what it expected, the TD error is high. A high TD error means that the agent can learn a lot from that experience since it indicates a gap in understanding. In our implementation, each memory item is given a priority proportional to its TD error (plus a small epsilon for fairness). When drawing mini-batches of experience to train on, we sample more frequently those transitions with larger errors.

By focusing on the "mistakes", the agent corrects its error faster. In practice, this prioritized replay made our robot learn efficient paths with fewer samples, because it spent more time on the rare but important events that taught it how to handle difficult situations. This implementation is an improvement over uniform replay that speeds up learning and makes better use of each training experience (A. A. Li et al., 2021). We still maintain a degree of randomness to avoid over-focusing on a small set of experiences, but overall this technique really boosted the learning efficiency of our Q-learning agent.

3.4.3 Soft Updates (Target Network)

To keep our training stable, we utilize a target network alongside the main Q-network as is common in DQN methods. However, instead of updating this target network by completely replacing it periodically with the main network, we applied soft updates. With soft update means that we gradually blend the main network's weights into the target network over time, rather than do it all at once (Andrychowicz et al., 2018). For example, after each training cycle, the target network might take 5% of the change from the main network (this corresponds to a decay factor of 0,95 on the old target wights) Andrychowicz et al., 2018. This way, the target network slowly tracks the learned Q-network.

The reason why we use soft updates is to avoid destabilizing the learning with a suddenly moving target. If the target network is updated too suddenly, the agent's foundation for estimating future rewards jump around, leading to possible oscillation or divergence in training. In contrast, a slow update makes the target network a smoother, gradually updated version of the main network. (Andrychowicz et al., 2018). This will lead to a more stable reference point for the DDQN updates. In our implementation, this led to a more consistent learning where the Q-values did not swing as much after target network updates, and the training curves were smoother.

3.4.4 QMIX Implementation

To extend our Q-learning approach to multi-agent scenarios, we implemented QMIX following the architecture described in Rashid et al., 2020. This allows agents to learn a joint action-value function while still being able to execute their policies independently.

Network Architecture Our implementation consists of three key components:

- **Agent Q-networks**: Each agent uses the same Q-network (thanks to parameter sharing) that processes local observations (80x5x5 tensor, thanks to frame stacking and our observation space) and outputs Q-values for each action.
- **Mixing network**: A feedforward neural network that combines the individual Q-values from all agents into a joint team Q-value (Q_{tot}). The mixing network uses hypernetworks to generate its weights, ensuring that weights remain non-negative to maintain the monotonicity constraint.

- **Hypernetworks**: Small neural networks that take the global state as input and output weights for the mixing network.

Training Process The training process follows the CTDE paradigm. During training, we utilize global state information to train the mixing network, allowing it to learn optimal coodination patterns between agents. Experiences are stored in a replay buffer, each entry storing tuples of $(o_i, a_i, r_i, o'_i, d_i, s_i, s'_i)$ representing the local observations, actions, rewards, next observations, done flags, global state, and next global state. When executing, each agent uses only its local observations to select actions, without access to the global state.

We use the smooth_l1 loss, which is a huber loss function that is less sensitive to outliers than the standard L2 loss. The loss is computed as follows:

$$\mathcal{L} = \sum_{i=1}^{n} \text{smooth\_l1}(Q_i(s_i, a_i), r_i + \gamma Q_{tot}(s'_i, a'_i)) \tag{3.1}$$

We also went with AdamW as our optimizer, which is a variant of Adam that includes weight decay regularization. This helps to prevent overfitting and improves generalization (Loshchilov et al., 2019).

3.4.5 Curriculum Learning

In our implementation, we used a curriculum learning approach to gradually increase the complexity of the training environment. We used three stages:

- **Stage 1 - Single Agent**: Training a single agent in a small grid (5x5) with no obstacles. This allows the agent to learn basic navigation and task completion without interference from other agents.
- **Stage 2 - Static Obstacles**: Slightly larger grid (10x8) with 16 shelves and 1 agent. This introduces static obstacles, allowing the agent to learn to navigate around them.
- **Stage 3 - Full Environment**: The full dynamic warehouse environment (34x32) with multiple agents (6) and dynamic obstacles (10 humans). The environment also has maximum static obstacle density (320 shelves). This stage allows the agent to learn to navigate in a complex environment with multiple agents and dynamic obstacles.

We manually moved on to the next stage after the agent no longer improved in the previous stage. This approach allowed us to gradually increase the complexity of the environment, making it easier for the agent to learn and adapt to

new challenges without being overwhelmed by the full complexity from the start.

3.4.6 Hyperparameter Tuning

We sadly didn't have time nor resources to do a full hyperparameter search, but we did try to manually tune the most important hyperparameters. We ended up using the following values:

Hyperparameter	**Value**
Learning Rate	0.00025
Discount Factor	0.99
Epsilon Start	1.0
Epsilon End	0.1
Epsilon Decay	0.995
Hidden Layer Size	(256, 128)
Buffer Size	200000
Batch Size	64
Soft Update Factor	0.0005

Table 3.1: Hyperparameters used for training.

These values were chosen based on our experiments and previous literature.

Chapter 4

Results

In this chapter, we present the results of our training and evaluation of the Q-learning algorithm against the A* algorithm. We first describe the training process and the environment setup, followed by a comparative analysis of both algorithms across various scenarios.

4.1 Q-learning Training

This section presents the training results of the Q-learning algorithm. The training was conducted in an environment with a grid size of 34x32, 6 robots, 10 humans, 320 shelves (full density), and 8 dropoff points, see 4.1. We evaluated the performance of the Q-learning algorithm every 200 training episodes, and the results are shown in Figure 4.2. We saved the model with the best performance, since we noticed performance degradation in later episodes.

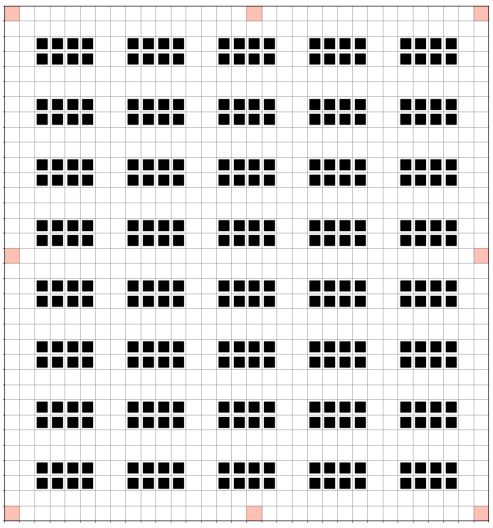


Figure 4.1: Warehouse grid (size 34x32) with dropoff points marked as red squares

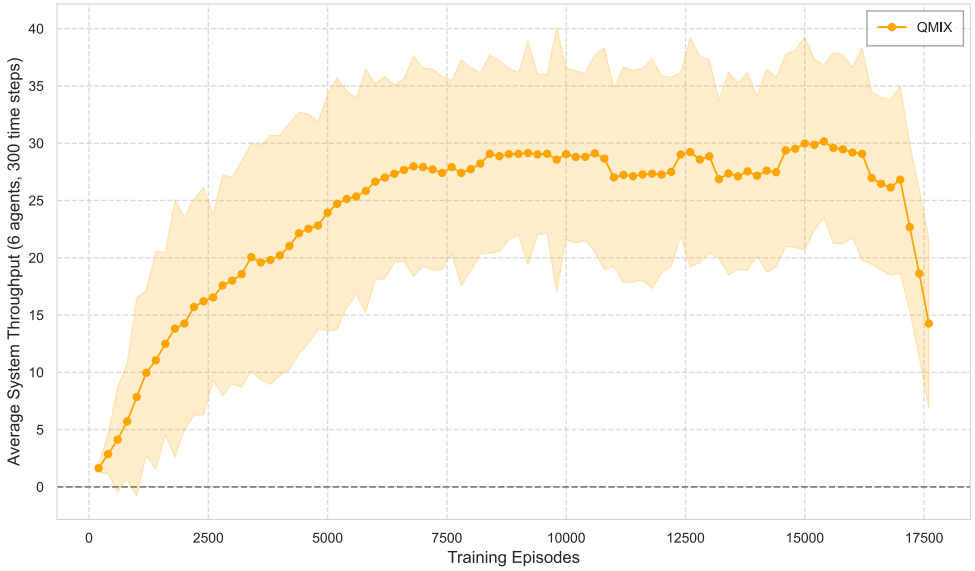


Figure 4.2: Q-learning (labeled as QMIX) average total throughput (6 robots) over 1000 time steps, measured every 200 training episodes. Smoothed over 7 episodes. Shaded area represents one standard deviation.

4.2 A* vs Q-learning Results

This section presents a comparative analysis of the A* and Q-learning algorithms across various warehouse scenarios. All experiments were conducted with a default configuration, where we then varied one parameter at a time to assess its impact on the performance of both algorithms.

Experimental Setup

All experiments maintained identical parameters except for the specific variable being tested in each scenario unless otherwise mentioned. The warehouse environment consisted of a 34×32 grid with 10 robotic agents navigating amongst 320 shelves (full capacity). Each configuration included 8 dropoff locations, and simulations were executed for 1000 time steps.

4.2.1 Impact of Dynamic Obstacles

We first examine how the presence of dynamic obstacles (human workers) affects algorithm performance. Figure 4.3 illustrates the relationship between the number of dynamic obstacles and agent throughput for both algorithms.

The results demonstrate that the Q-learning approach consistently outperforms A* across all tested obstacle densities, maintaining approximately 30% higher throughput. Both algorithms show a linear decrease in performance as the number of dynamic obstacles increases. Figure 4.4 isolates the performance advantage without the distraction of overall declining trends, showing that the absolute difference between algorithms remains between 4-5 tasks per agent regardless of obstacle count. Interestingly, the gap slightly widens as the obstacle count increases from 0 to 20, then stabilizes around 5 tasks per agent for higher obstacle densities. This suggests that Q-learning's multi-agent coordination strategy provides greater advantage in the initial stages of obstacle introduction, but both algorithms face similar challenges in high-density scenarios.

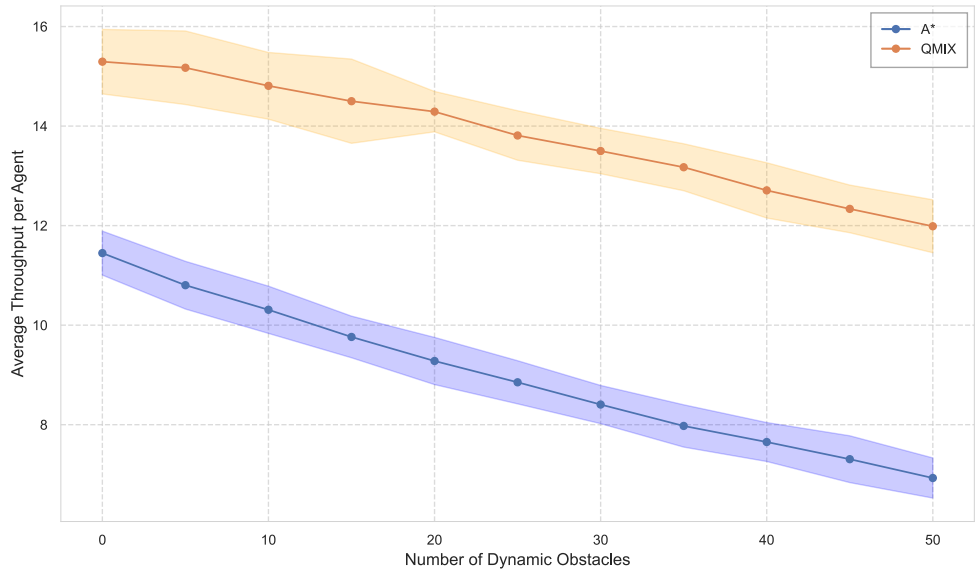


Figure 4.3: Comparison of A* and Q-learning (labeled as QMIX) algorithm performance with increasing numbers of dynamic obstacles. Average throughput per agent measured over 1000 simulation steps; shaded areas represent one standard deviation.

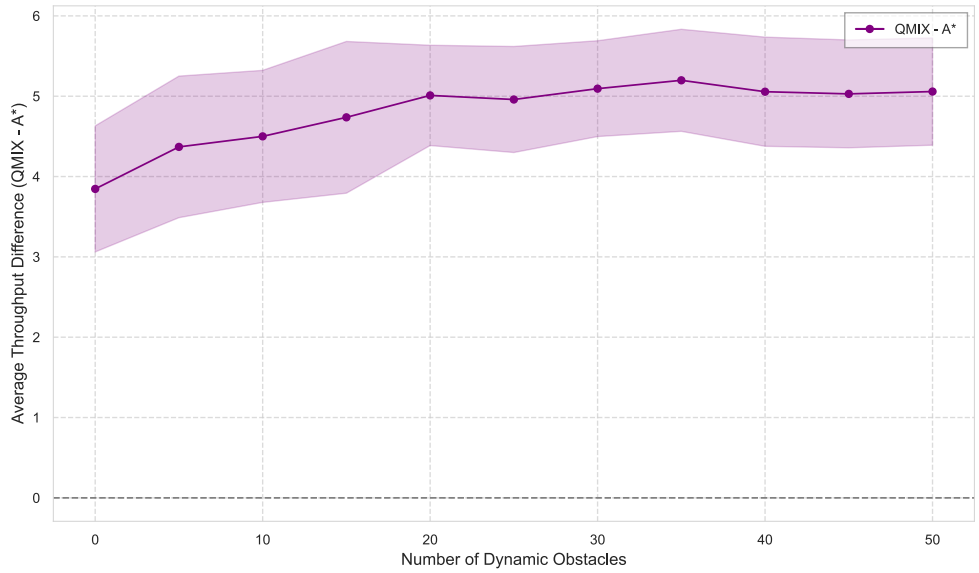


Figure 4.4: Performance difference between Q-learning (labeled as QMIX) and A* algorithms with increasing numbers of dynamic obstacles. Positive values indicate Q-learning's performance advantage; shaded area represents combined standard deviation of both algorithms.

4.2.2 Impact of Static Obstacles

Next, we examine how static obstacles (shelves) influence algorithm performance. Figure 4.5 reveals a clear contrast in performance between A* and the Q-learning approach as the number of static obstacles varies.

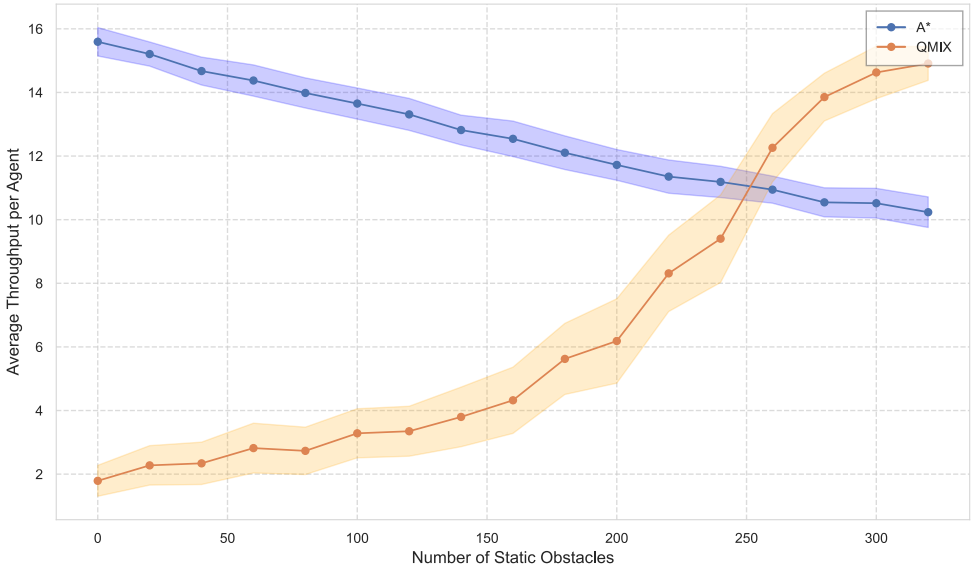


Figure 4.5: Comparison of A* and Q-learning (labeled as QMIX) algorithm performance with increasing numbers of static obstacles (shelves). The performance trends move in opposite directions, with a crossover point at approximately 250 shelves.

The results demonstrate a fundamental difference in how A* and the Q-learning approach respond to environmental structure. A* exhibits an expected linear decrease in performance as the number of shelves increases, while the Q-learning approach shows a non-linear increase in throughput as shelf density increases, from approximately 2 tasks per agent at 0 shelves to 15 tasks per agent at 320 shelves.

This behaviour can partially attributed to the Q-learning's training environment, which featured high shelf density. As the environment approaches that which the agents were trained on, the networks are able to leverage their training to improve performance. This shows a weakness in the generalisation ability of our Q-learning implementation, as it is unable to adapt to the low shelf density environment.

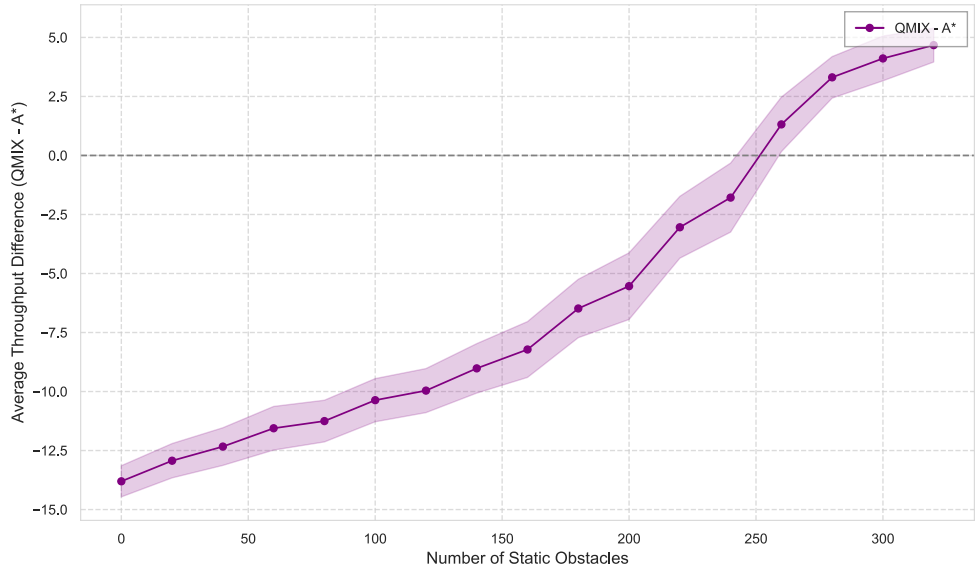


Figure 4.6: Performance difference between Q-learning (labeled as QMIX) and A* algorithms with increasing numbers of static obstacles (shelves). Positive values indicate Q-learning's performance advantage; shaded area represents combined standard deviation of both algorithms.

4.2.3 Impact of Robots

We also investigated how the number of robots affects algorithm performance. Figure 4.7 illustrates the performance of both algorithms as the number of robots increases.

We observe that both algorithms exhibit a linear decrease in performance as the number of robots increases, with the Q-learning approach consistently outperforming A*. Of note here is the stable standard deviation of A* across all robot counts, in contrast to Q-learning's increasing standard deviation. There could be several reasons for this, including that A* has a central planner that helps coordinate the robots, while the Q-learning approach relies on policies. It's likely also affected by the fact that the Q-learning agent is trained in an environment with a fixed number of robots, and so doesn't generalise well to environments with large amounts of them. Thinking about it, it makes sense that highly congested environments would have a higher variance in performance, as the robots are more likely to get stuck.

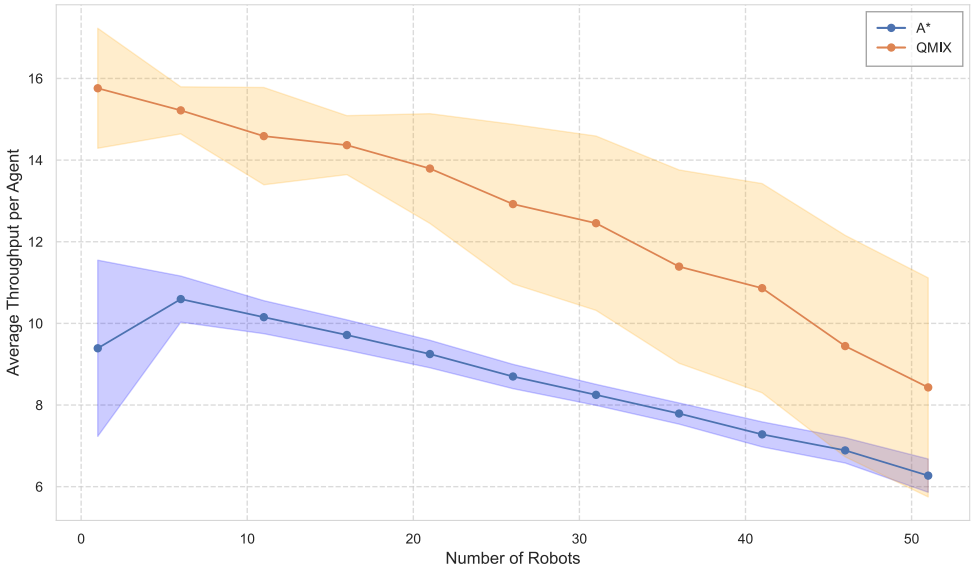


Figure 4.7: Comparison of A* and Q-learning (labeled as QMIX) algorithm performance with increasing numbers of robots.

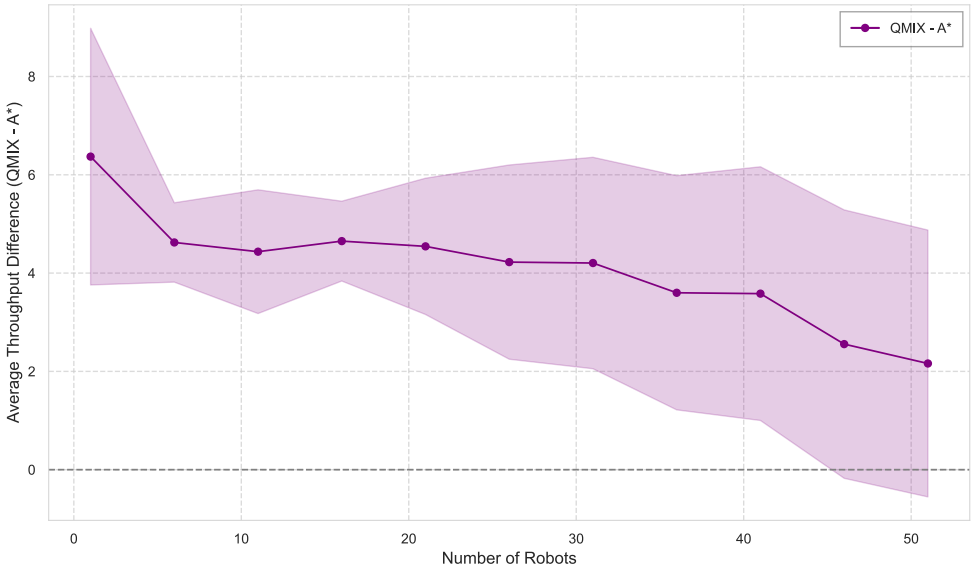


Figure 4.8: Performance difference between Q-learning (labeled as QMIX) and A* algorithms with increasing numbers of robots. Positive values indicate Q-learning's performance advantage; shaded area represents combined standard deviation of both algorithms.

4.2.4 Impact of Grid Sizes

Finally, we examined how the size of the grid affects algorithm performance. Figure 4.9 shows the performance of both algorithms as the grid size increases.

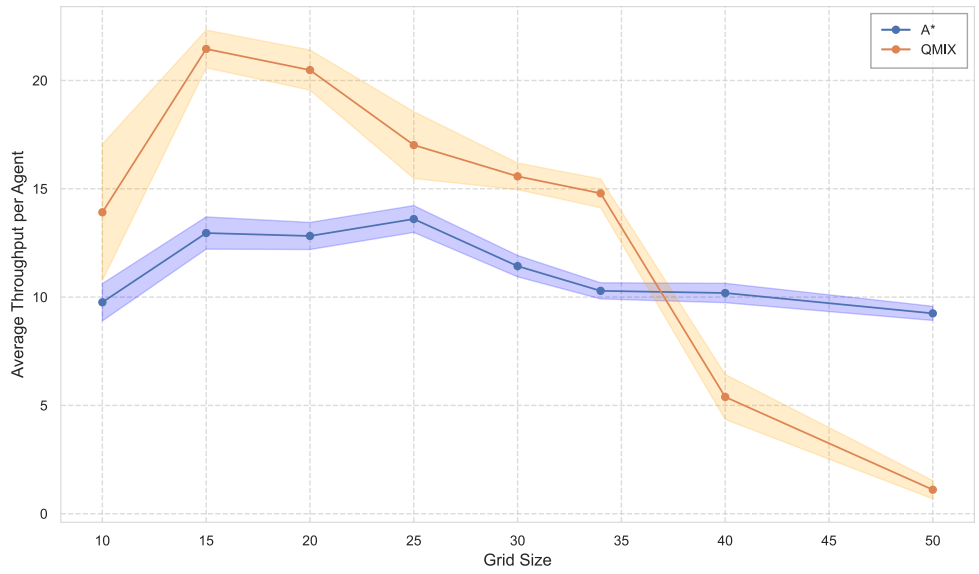


Figure 4.9: Comparison of A* and Q-learning (labeled as QMIX) algorithm performance with increasing square (n^2) grid size.

In these figures, we observe that A* keeps a relatively consistent performance across all grid sizes, while Q-learning initially outperforms A* but then drops below it as the grid size increases to sizes larger than it was trained on. Unlike the robot count, where the standard deviation of Q-learning increased with the number of robots, we see very tight bands for both algorithms across all grid sizes. This is in line with increasing something like shelves; an increase slightly lowers throughput, but the stationary nature gives rise to little variance in performance.

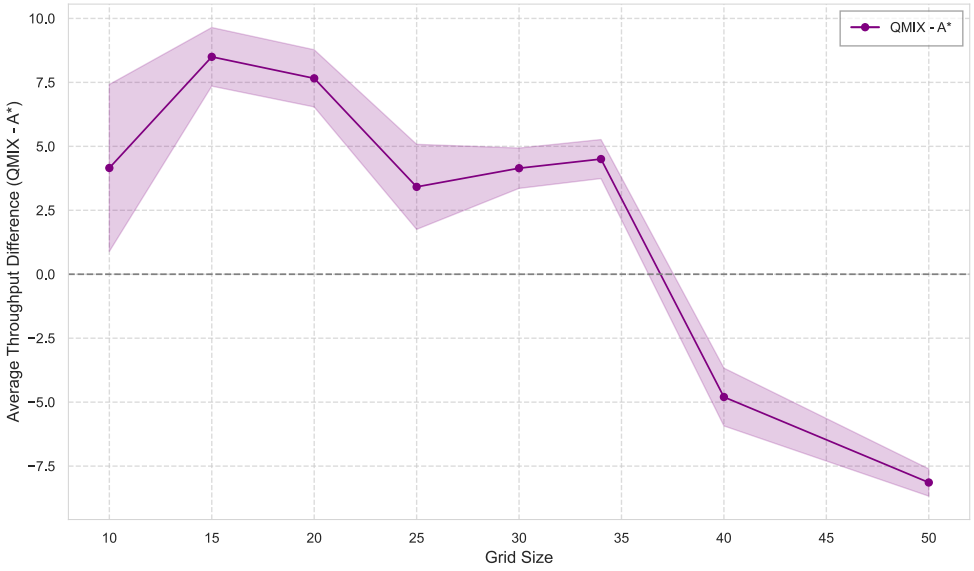


Figure 4.10: Performance difference between Q-learning (labeled as QMIX) and A* algorithms with increasing square (n^2) grid size. Positive values indicate Q-learning's performance advantage; shaded area represents combined standard deviation of both algorithms.

Chapter 5

Discussion

5.1 Challenges

The implementation of the Double Q-network (DDQN) in a dynamic warehouse setting was significantly more complex than implementing the classical A* pathfinding algorithm. A* is a well-established path search method with relatively straightforward implementation and predictable behavior, unlike the modified Q-learning approach. A basic DQN already requires mechanisms like experience replay and a target network to stabilize learning. These components are complex and harder to debug potential problems.

Beyond the algorithmic components, tuning a Q-learning agent involved a lot of trial and error. There are many hyperparameters (replay buffer size, learning rate, exploration schedule, target update frequency, etc) that interact in non-obvious ways. Getting the network to converge to a stable behavior required adjusting these parameters iteratively. This hands-on tuning aspect is common in deep reinforcement learning development (Vasan et al., 2024).

Defining an effective reward function for the Q-learning agent was another challenge. A naive reward schema might only give a positive reward upon reaching the goal and perhaps a negative reward for collisions. Such a reward schema would make learning extremely difficult, since the agent receives little feedback for most of its steps. In our project, we observed that an agent trained with only a goal-completion reward struggled to make progress; it would often wander in a random manner or get stuck. This was because it rarely completed a task, hence no reward was given.

To address this, we experimented with reward shaping, adding an incentive for the agent to walk in the correct direction. For instance, we gave small step penalties to encourage shorter paths, and rewards for moving closer to the

goal. This did indeed help the agent to navigate to its goal. However, these rewards also required a lot of trial and error before getting it right. In our case, we had problems with agents oscillating and walking around the checkpoint to accumulate rewards or avoid penalties, without making real progress toward the goal. We had to revise our reward design multiple times to ensure that the agent was being driven toward the intended goal (fast, collision-free navigation) and not getting stuck or making unnecessary moves.

5.2 Extensions to Q-learning

The Q-learning approach required multiple extensions in order to perform in our environment. We implemented several extensions beyond the vanilla algorithm. The extensions that we implemented were the following: **DDQN, frame stacking, prioritized experience replay, soft target network updates, QMIX, and Parameter sharing** Beyond the techniques we implemented, there are numerous other extensions we considered from recent research in RL. For example, the **dueling network architecture** is one such improvement. This technique separates the estimation of state value and advantage for each action, which can speed up learning by better generalizing the value of different actions in similar states (Z. Wang et al., 2016). Another extension is using **Noisy Nets** for exploration, which injects learned noise into the network weights to encourage exploration instead of using an ϵ - greedy policy (Fortunato et al., 2019). Another extension we thought about is **distributional Q-learning**, where the network predicts a distribution of possible returns instead of just the mean (Bellemare et al., 2017).

Rainbow's success illustrates that these extensions tend to be complementary; each adds some performance or stability benefit. For our project, due to time constraints, we implemented a subset of such improvements (as described above). Still, reflecting on Rainbow, one can imagine that implementing all of those techniques might further improve our Q-learning's performance in the warehouse. The downside is the complexity, every new extension adds more hyperparameters and interactions to manage. Therefore we only added the most impactful features for our domain (like multi-agent coordination via QMIX and replay enhancements for sample efficiency). Overall, the extensions we chose did boost the Q-learning agent's learning ability and final performance, bringing it closer to a level that could compare with the A* algorithm.

5.3 Analysis of performance trends

In dynamic obstacle scenarios 4.3, both the A* and Q-learning algorithm decreases in performance (throughput) linearly as more human obstacles are introduced. This is an expected behavior due to the congestion and replanning needs. The Q-learning algorithm consistently maintains a higher throughput, about 30% above the A*, across all the tested obstacle counts. This is also an expected behavior since the Q-learning approach is known for its adaptable behavior in dynamic environments. Figure 4.4 further highlights this gap, where the Q-learning approach delivers about 4-5 more packages than the A* algorithm; this gap widens as the obstacle count increases up to 20 humans. This indicates that the Q-learning approach works better at first when obstacles start showing up, but when there are lots of obstacles, both methods struggle with a lot of traffic. Figure 4.7 demonstrates similar results; both algorithms decreases in performance linearly as more agents are added.

In the static obstacle scenario (4.5 the A* performance was expected, decreasing linearly as more shelves were placed in the warehouse, since longer detours are needed around obstacles. Interestingly, the performance of the Q-learning algorithm was very poor with no to little shelves, but the performance then increased as the number of shelves increased. There is a crossover point around 240-250 shelves where the Q-learning approach overtakes the A* algorithm. In other words, the Q-learning approach performs poorly in an open warehouse (without static obstacles) but gets better when obstacles are added, whereas A* behaves in the opposite manner. Another interesting finding is that when the maximum number of shelves is added, the Q-learning agent performs better than the A* agent in terms of throughput. We actually expected the A* to perform better or at least as good as the Q-learning agent in this scenario, due to the A*'s ability to navigate in a near-optimal manner within a static environment. One possible reason why the A* did not perform as good as the Q-learning agent could be that our A* implementation is not optimal.

Our hypothesis to why the Q-learning approach performs poorly when static obstacles are removed is that the Q-learning agents overfitted to the training environment, which consisted of a high shelf density. The DDQN network seems to have picked up on specific features of the environment, like using shelves to help navigate or learning patterns that only work with a cluttered warehouse. This is a common problem in deep reinforcement learning, where policies can memorize quirks of the training simulator rather than learning truly general skills (Zhang et al., 2018). In the report by Zhang et al., 2018, it is discussed how RL agents often show limited generalization, especially

in deterministic simulated environments where the lack of noise and variation can lead to overfitting (Zhang et al., 2018). They found that introducing greater diversity during training, for example, randomizing the environmental conditions, is crucial for robust performance. In our training, the static layout was fixed; therefore, the Q-learning agents probably over-specialized to that specific layout. Thus, our result confirms a classic pitfall in RL, that a trained policy can have very good results in its comfort zone but may have very bad results outside it. With that being said, our policy could have been better if we had introduced greater diversity during our training.

When varying the amount of robots 4.7, we can see our Q-learning approach is handling it well, consistently outperforming A*. For both A* and the Q-learning approach, there's a clear declining trend, likely caused by congestion and competition for key points like dropoffs. The performance of the Q-learning approach appears to be dropping slightly faster than that of A* (see 4.8), which might once again be caused by the fact that it was never trained on extremely dense environments. Of note is the widening of the standard deviation band for Q-learning in contrast to the stable one of A*; a deterministic algorithm will be more reliable than one based on policies, and policies will have a harder time dealing with congestion.

Lastly, we have 4.9, where A* has very stable performance across all grid sizes. The Q-learning approach struggles to perform in grids much larger than the one it was trained on, but outperforms A* on the ones close to its training environment, including the smallest 10x10 one. Something of note here is that for most of these scenarios, the computational complexity would create huge issues for A* in real-world scenarios, while a Q-learning approach would simply run inference on each robot, making it a more scalable solution.

In a broader perspective, these findings highlight a key trade-off between classical planning and learning-based methods. The Q-learning agent performed good in scenarios that closely resembled its training distribution, but struggled when faced with conditions outside its experience. A* offered a steadier, more predictable performance across a range of conditions due to its algorithmic guarantees, yet it lacked the ability to adapt to dynamic environments. This suggest that in a relatively fixed and structured environment, a method like A* provides reliability and simplicity, whereas in highly dynamic or uncertain settings, a learning-based approach (provided it has been trained on sufficient diversity) can provide superior results by leveraging its experience. Additionally, the decentralized execution of our Q-learning approach solution means it scales well with more agents and can adjust on the fly to local changes, in contrast to A*, which requires a centralized planner

that may become a bottleneck as complexity grows. However, Q-learning's reliance on training data implies that maintaining its performance within new environments would require retraining, whereas A* can handle new layouts or obstacle configurations on the fly. In summary, our comparison underlines the classical exploration-exploitation dilemma in a multi-agent context; the RL approach exploited specific environmental structure to achieve high throughput when conditions were familiar, while the A* approach provided more robust performance in unfamiliar scenarios.

5.4 Reflection on initial assumptions

Our initial assumption was that a learning-based approach would perform better than A* in dynamic environments. This is indeed confirmed by the dynamic obstacle experiments, where the Q-learning agent clearly outperformed A*, confirming the idea that RL can handle unpredictability better than static planning. We also assumed that the A* algorithm would show great results in a static environment, which it did. However, one outcome we did not expect was just how strongly the Q-learning algorithm was affected when the static obstacles were removed. We expected that the learned policy to at least match A* in an empty warehouse (since, intuitively, fewer obstacles should make navigation easier for an agent). In reality however, our policy was closely tied to its training conditions, which limited its flexibility to adapt to other conditions.

5.5 Limitations and future work improvements

It is important to acknowledge the limitations of our study, as they provide balance to the conclusions and highlight ways to improve. A primary limitation is the limited diversity of our training scenarios. Our Q-learning agents were trained in a warehouse layout with a fixed grid size and a fixed number of shelves. This likely caused the policy to over-specialize to that specific layout. In real warehouses, static obstacles such as shelves can vary, so an RL agent trained only to one specific layout may not perform well elsewhere. Future work should include training diversity by training the agents in different environments. Such as randomizing the static obstacles and even changing the entire layout of the grid. Another notable limitation lies in the implementation of the A* algorithm itself. Our tests show that the Q-learning agent actually outperformed the A* algorithm in a completely static multiagent environment, in terms of throughput. Since A* is a classical search algorithm that is expected

to find the shortest and most optimal paths in static environments, this result is surprising and indicates that our implementation might not be optimal. In theory, the A* algorithm should at least match, or even outperform our Q-learning agent in terms of throughput. This performance issue could be because of the way we implemented certain parts of A*, such as how replanning is handled, or maybe how agents manage the reservation of time and space, or something else. A good follow-up on this paper would be to compare the Q-learning approach to another more robust multi-agent path-planning algorithm like D* Lite. Another limitation is that our evaluation metrics focused on throughput; we did not deeply analyse other aspects, like collision rates or path-finding optimality. It is possible that the Q-learning agent achieved higher throughput at the cost of more frequent near-misses or dangerous maneuvers, which were not captured in our metrics. Evaluating safety and reliability is something worth digging into for future works, especially for real-world applicability. Computational complexity is another point that would greatly affect the choice of algorithm, since a big perk of the RL choice is the completely decentralised execution, unlike the A* algorithm, which requires occasional central planning. Lastly, our implementation of the Q-learning algorithm itself could be a limitation since all of our hyperparameters could potentially be modified to create better policies for the agents. We did not have the time nor resources to perform a proper hyperparameter search.

5.6 Ethical Considerations

The implementation of autonomous warehouse robots raises important ethical questions that must be carefully considered alongside the technical performance metrics evaluated in this study.

5.6.1 Workforce Impact and Automation

The primary ethical concern surrounding warehouse automation is the potential displacement of human workers. Traditional warehouse operations rely heavily on manual labor for tasks such as order picking, inventory management, and material handling. The introduction of autonomous robots performing these tasks could lead to significant job losses, particularly affecting workers in lower-skilled positions who may have limited opportunities for retraining or career transition.

However, this technological shift also presents opportunities for workforce evolution. While robots may replace certain manual tasks, they can also cre-

ate new job categories requiring different skill sets, such as robot maintenance, system monitoring, and advanced logistics coordination. The key ethical imperative is ensuring that the benefits of automation are shared equitably, with adequate support provided for displaced workers through retraining programs, transition assistance, and social safety nets.

Organisations implementing such systems have a responsibility to consider the social impact of their technology choices. This includes transparent communication with affected workers, investment in retraining programmes, and gradual implementation that allows for workforce adaptation rather than abrupt displacement.

5.6.2 Safety and Human-Robot Interaction

Another critical ethical consideration is the safety of human workers who continue to operate alongside autonomous robots. Our study focused on throughput optimisation, but real-world implementation must prioritise human safety above efficiency gains. This includes ensuring that robots can safely navigate environments with human workers, implementing fail-safe mechanisms, and maintaining clear protocols for human-robot interaction.

The algorithms evaluated in this study demonstrate different approaches to handling dynamic obstacles, including human workers. While our Q-learning approach showed superior adaptability to dynamic environments, the ethical responsibility extends beyond algorithmic performance to include comprehensive safety testing, robust sensor systems, and conservative decision-making in uncertain situations.

5.6.3 Transparency and Explainability

An important ethical consideration in deploying learning-based systems like Q-learning in warehouse environments is the "black box" nature of neural networks. Unlike A* pathfinding, which follows clear logical steps that can be traced and understood, the decision-making process of a trained Q-learning agent is opaque. This lack of transparency can be problematic when:

- Investigating incidents or near-misses involving robotic systems
- Ensuring compliance with safety regulations that may require explainable automated decisions
- Building trust between human workers and automated systems

- Debugging unexpected behaviours in operational environments

Future implementations of learning-based warehouse systems should consider incorporating explainable AI techniques to make robot decision-making more transparent and accountable.

5.7 Sustainability Considerations

The environmental impact of warehouse automation extends beyond immediate operational efficiency to encompass broader sustainability concerns that are increasingly important in corporate decision-making and environmental stewardship.

5.7.1 Energy Consumption and Carbon Footprint

While our study focused on throughput optimisation, the energy consumption of different algorithmic approaches represents a critical sustainability factor. The computational requirements of our two approaches differ significantly:

The A* algorithm requires centralised computation for path planning, which involves periodic intensive calculations to determine optimal routes for all robots. This centralised approach may lead to higher peak energy consumption during planning phases, but the computational load is predictable and can be optimised through efficient hardware utilisation.

In contrast, the Q-learning approach distributes computational load across individual robots, each running neural network inference continuously. While this decentralised approach may offer better scalability, it requires constant energy consumption for inference on each robot. The training phase of Q-learning also involves significant computational energy expenditure, though this is a one-time cost that can be amortised over the system's operational lifetime.

Future research should include comprehensive energy consumption analysis to understand the long-term sustainability implications of different algorithmic choices. This analysis should consider not only operational energy consumption but also the capex costs of specialized hardware requirements and the energy costs of algorithm development and training.

5.7.2 Resource Efficiency and Waste Reduction

Autonomous warehouse systems can contribute to sustainability through improved resource efficiency. More efficient path planning and coordination can

reduce unnecessary robot movements, leading to lower energy consumption and reduced wear on mechanical components. This efficiency can extend the operational lifetime of robotic systems and reduce the frequency of hardware replacement.

The improved throughput demonstrated by our Q-learning approach in dynamic environments could translate to better space utilization and reduced need for warehouse expansion, contributing to more sustainable land use and reduced construction-related environmental impact.

5.7.3 Lifecycle Considerations

The sustainability assessment of warehouse automation must consider the complete lifecycle of robotic systems, from manufacturing through operation to end-of-life disposal. The choice of algorithms can influence hardware requirements, with implications for the environmental impact of manufacturing and the longevity of deployed systems.

The adaptability of learning-based approaches like Q-learning could potentially extend the useful lifetime of robotic systems by allowing them to adapt to changing operational requirements without hardware modifications. Conversely, the deterministic nature of A* may provide more predictable performance degradation patterns, facilitating better maintenance scheduling and resource planning.

5.7.4 Broader Environmental Impact

The efficiency gains from optimised warehouse operations can contribute to broader environmental benefits through reduced transportation needs, improved inventory management, and more efficient supply chain operations. However, these benefits must be weighed against the environmental costs of increased automation infrastructure and the potential for increased consumption enabled by greater efficiency.

Organisations implementing these technologies have a responsibility to consider the net environmental impact of their automation choices, including both direct operational effects and indirect consequences for consumption patterns and supply chain behaviour.

Chapter 6

Conclusion

The aim of this thesis was to investigate how the A* and Q-learning algorithms compare in terms of throughput for robot path planning in dynamic warehouse environments. Our analysis showed that while both algorithms have their own strengths, their effectiveness depends on the environmental context. The Q-learning algorithm, specifically implemented using the Double Deep Q-Network (DDQN) with a mixing network (QMIX), performed better in dynamic scenarios with lots of human obstacles. This confirmed our initial assumption that reinforcement learning would do well in unpredictable situations because it can adapt and learn how to coordinate in such environments. On the other hand, the classic A* algorithm was more reliable and consistent across various static configurations, showing its strengths in static environments. A noteworthy discovery was that the Q-learning agent's performance dropped a lot in scenarios without static obstacles, showing the challenges of generalizing an RL agent's learned behavior.

Overall, this study successfully demonstrated that reinforcement learning is a viable option for robot coordination and throughput in complex, dynamic environments, providing valuable insights for future improvements and real-world use in warehouses. Future work should focus on improving the Q-learning agent's generalization capabilities, exploring more diverse training environments, and comparing it against other advanced multi-agent path-planning algorithms. Additionally, ethical and sustainability considerations should be integrated into the design and deployment of autonomous warehouse systems to ensure responsible and beneficial use of these technologies.

Bibliography

Albrecht, Stefano V., Filippos Christianos, and Lukas Schäfer (2024). *Multi-agent reinforcement learning: Foundations and modern approaches*. Cambridge, MA: MIT Press.

Andrychowicz, Marcin et al. (2018). *Hindsight experience replay*. arXiv: 1707.01495 [cs.LG].

Bellemare, Marc G., Will Dabney, and Rémi Munos (2017). *A distributional perspective on reinforcement learning*. arXiv: 1707.06887 [cs.LG].

Brightpick (2024). *Leading sports nutrition retailer The Feed picks 50,000 items per day with Brightpick Autopicker*. https://brightpick.ai/resources/the-feed/ (visited on 07/03/2025).

CognitOps (2023). *Warehouse picking strategies: Your ultimate guide to boosting efficiency*. https://cognitops.com/warehouse-picking-strategies/ (visited on 07/03/2025).

de Koster, René (2018). "Automated and robotic warehouses: Developments and research opportunities". In: *Logistics and Transport* 38.4, p. 33. DOI: 10.26411/83-1734-2015-2-38-4-18.

de Koster, René, Tho Le-Duc, and Kees Jan Roodbergen (2007). "Design and control of warehouse order picking: A literature review". In: *European Journal of Operational Research* 182.2, pp. 481–501. ISSN: 0377-2217. DOI: 10.1016/j.ejor.2006.07.009.

Dorigo, Marco, Mauro Birattari, and Thomas Stutzle (2006). "Ant colony optimization". In: *IEEE Computational Intelligence Magazine* 1.4, pp. 28–39. DOI: 10.1109/MCI.2006.329691.

Fortunato, Meire et al. (2019). *Noisy networks for exploration*. arXiv: 1706.10295 [cs.LG].

Hart, Peter E., Nils J. Nilsson, and Bertram Raphael (1968). "A formal basis for the heuristic determination of minimum cost paths". In: *IEEE Transactions on Systems Science and Cybernetics* 4.2, pp. 100–107. DOI: 10.1109/TSSC.1968.300136.

Hashemi-Pour, Cameron (2023). *e-commerce*. https://www.techtarget.com/searchcio/definition/e-commerce (visited on 07/03/2025).

Hasselt, Hado van, Arthur Guez, and David Silver (2016). "Deep reinforcement learning with double Q-learning". In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*. AAAI Press, pp. 2094–2100.

Kennedy, J. and R. Eberhart (1995). "Particle swarm optimization". In: *Proceedings of ICNN'95 - International Conference on Neural Networks*. Vol. 4, pp. 1942–1948. DOI: 10.1109/ICNN.1995.488968.

Koenig, Sven and Maxim Likhachev (2002). "D*lite". In: *Eighteenth National Conference on Artificial Intelligence*. USA: American Association for Artificial Intelligence, pp. 476–483.

LaValle, Steven M. (1998). *Rapidly-exploring random trees: A new tool for path planning*. Technical Report TR 98-11. Computer Science Department, Iowa State University.

Lee, Hung-Yu and Chase C. Murray (2019). "Robotics in order picking: evaluating warehouse layouts for pick, place, and transport vehicle routing systems". In: *International Journal of Production Research* 57.18, pp. 5821–5841. DOI: 10.1080/00207543.2018.1552031.

Li, Ang A., Zongqing Lu, and Chenglin Miao (2021). *Revisiting prioritized experience replay: A value perspective*. arXiv: 2102.03261 [cs.LG].

Li, Keqin et al. (2024). *Research on reinforcement learning based warehouse robot navigation algorithm in complex warehouse layout*. arXiv: 2411.06128 [cs.RO].

Lin, Long-Ji (1992). "Self-improving reactive agents based on reinforcement learning, planning and teaching". In: *Machine Learning* 8, pp. 293–321. DOI: 10.1007/BF00992699.

Loshchilov, Ilya and Frank Hutter (2019). *Decoupled weight decay regularization*. arXiv: 1711.05101 [cs.LG].

Lowe, Ryan et al. (2017). "Multi-agent actor-critic for mixed cooperative-competitive environments". In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., pp. 6382–6393.

Mnih, Volodymyr et al. (2015). "Human-level control through deep reinforcement learning". In: *Nature* 518.7540, pp. 529–533. DOI: 10.1038/nature14236.

Mnih, Volodymyr et al. (2013). *Playing Atari with deep reinforcement learning*. arXiv: 1312.5602 [cs.LG].

Rashid, Tabish et al. (2020). “Monotonic value function factorisation for deep multi-agent reinforcement learning”. In: *Journal of Machine Learning Research* 21.178, pp. 1–51.

Schaul, Tom et al. (2015). *Prioritized experience replay*. arXiv: 1511.05952 [cs.LG].

Schulman, John et al. (2017). *Proximal policy optimization algorithms*. arXiv: 1707.06347 [cs.LG].

Sunehag, Peter et al. (2018). “Value-decomposition networks for cooperative multi-agent learning based on team reward”. In: *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*. Richland, SC: International Foundation for Autonomous Agents and Multiagent Systems, pp. 2085–2087.

Sutton, Richard S. and Andrew G. Barto (2018). *Reinforcement learning: An introduction*. Cambridge, MA: A Bradford Book. ISBN: 0262039249.

Vasan, Gautham et al. (2024). *Revisiting sparse rewards for goal-reaching reinforcement learning*. arXiv: 2407.00324 [cs.RO].

Wang, Yutong et al. (2025). *LNS2+RL: Combining multi-agent reinforcement learning with large neighborhood search in multi-agent path finding*. arXiv: 2405.17794 [cs.RO].

Wang, Ziyu et al. (2016). *Dueling network architectures for deep reinforcement learning*. arXiv: 1511.06581 [cs.LG].

Watkins, Christopher John Cornish Hellaby (1989). “Learning from delayed rewards”. PhD thesis. United Kingdom: King’s College, Cambridge.

Zhang, Amy, Nicolas Ballas, and Joelle Pineau (2018). *A dissection of overfitting and generalization in continuous reinforcement learning*. arXiv: 1806.07937 [cs.LG].

Appendix A

Implementation Details

A.1 Code Repository

The complete implementation of this project, including all algorithms, environments, and evaluation code, is available as an open-source repository at:

https://github.com/vidarrio/kex25

TRITA-EECS-EX-2025:319
Stockholm, Sweden 2025

www.kth.se